

LESSON 9

SLAM & Path Planning

Simultaneous Localization And Mapping + how robots find their way.

- 9.1 SLAM Overview
- 9.2 Types of SLAM
- 9.3 Path Planning: A*, Dijkstra, RRT
- 9.4 A* Implementation
- 9.5 Underwater SLAM Challenges

9.1 What is SLAM?

SLAM = Simultaneous Localization And Mapping

The chicken-and-egg problem:

- To know WHERE I am, I need a MAP
- To build a MAP, I need to know WHERE I am
- SLAM solves BOTH at the same time!

Input: Sensor data (LiDAR, camera, sonar, IMU)

Output: Map of environment + robot's position in that map

Real-world analogy:

You wake up in an unknown building, blindfolded.

You can touch walls and count steps.

You must draw a map of the building

AND know where you are on that map

AT THE SAME TIME.

9.2 Types of SLAM

Type	How It Works	Used With
EKF-SLAM	Extended Kalman Filter. Tracks robot pose + landmarks in one state vector.	Small environments, few landmarks
Particle Filter (FastSLAM)	Many random guesses (particles). Each particle has its own map. Keep best.	Large environments, non-linear models
Graph SLAM	Poses = nodes. Constraints = edges. Optimize entire graph.	Most modern systems. Loop closure detection.
Visual SLAM (V-SLAM)	Camera images for feature detection & tracking.	ORB-SLAM, LSD-SLAM. Mono/stereo cameras.
LiDAR SLAM	Laser scans for precise mapping.	Hector SLAM, Gmapping, CARTOGRAPHER.

9.3 Path Planning Algorithms

DIJKSTRA'S ALGORITHM:

- Finds shortest path in WEIGHTED graph
- Explores in all directions equally
- Guaranteed optimal
- Like BFS but with a priority queue
- $O((V + E) \log V)$

A* (A-STAR):

- Dijkstra + HEURISTIC (estimated distance to goal)
- Explores in the direction of the goal first
- Much faster than Dijkstra when goal location is known
- $f(n) = g(n) + h(n)$
 - $g(n)$ = actual cost from start to n
 - $h(n)$ = estimated cost from n to goal (heuristic)
- Common heuristics:
 - Manhattan: $|x1-x2| + |y1-y2|$ (grid, 4-direction)
 - Euclidean: $\sqrt{(x1-x2)^2 + (y1-y2)^2}$ (any direction)

RRT (Rapidly-exploring Random Trees):

1. Pick random point in space
 2. Find nearest node in current tree
 3. Extend tree toward random point
 4. Check for collisions
 5. Repeat until goal reached
- Good for: high-dimensional spaces, complex obstacles
 - NOT optimal (path has kinks)

RRT*:

- RRT + rewiring for shorter paths
- Asymptotically optimal (gets better over time)
- Used in MoveIt2 for robot arm planning!

9.4 A* Implementation

```

import heapq

def a_star(grid, start, end):
    rows, cols = len(grid), len(grid[0])

    def heuristic(r, c):
        return abs(r - end[0]) + abs(c - end[1]) # Manhattan

    # Priority queue: (f_score, row, col)
    heap = [(heuristic(start[0],start[1]), 0, start[0], start[1])]
    # f_score, g_score, row, col
    visited = set()
    g_scores = {(start[0],start[1]): 0}
    parent = {}

    while heap:
        f, g, r, c = heapq.heappop(heap)

        if (r, c) == end:
            # Reconstruct path
            path = []
            pos = end
            while pos in parent:
                path.append(pos)
                pos = parent[pos]
            path.append(start)
            return path[::-1]

        if (r, c) in visited:
            continue
        visited.add((r, c))

        for dr, dc in [(0,1),(0,-1),(1,0),(-1,0)]:
            nr, nc = r+dr, c+dc
            if (0<=nr<rows and 0<=nc<cols
                and grid[nr][nc] != 1
                and (nr,nc) not in visited):
                new_g = g + 1
                if new_g < g_scores.get((nr,nc), float("inf")):
                    g_scores[(nr,nc)] = new_g
                    f = new_g + heuristic(nr, nc)
                    heapq.heappush(heap, (f, new_g, nr, nc))
                    parent[(nr,nc)] = (r, c)

    return None # no path

```

9.5 Algorithm Comparison

Algorithm	Optimal?	Complete?	Best For
BFS	Yes (unweighted)	Yes	Grid, equal costs
Dijkstra	Yes	Yes	Weighted graphs
A*	Yes (admissible h)	Yes	Weighted + known goal
RRT	No	Probabilistic	High-dim, complex

RRT*	Asymptotically	Probabilistic	Optimal + complex
------	----------------	---------------	-------------------